

Python Study Notes (Part 3)

These sections cover essential topics beyond the current code files. Each section uses the pattern: explanation → code → explanation.

Core Language Patterns — Unpacking, swapping, slicing

Python has concise, readable patterns for common tasks.

```
# Unpacking
first, *middle, last = [1, 2, 3, 4, 5]

# Swapping
a, b = 10, 20
a, b = b, a

# Slicing
nums = [0, 1, 2, 3, 4, 5]
print(nums[::2])    # every 2nd element
print(nums[::-1])  # reversed copy
```

Output:

```
[0, 2, 4]
[5, 4, 3, 2, 1, 0]
```

Unpacking collects the “rest” into a list. Tuple-style swapping avoids a temp variable. Slicing supports step sizes and reverse copies.

Data Model — Identity vs equality, mutability, hashability

Understanding Python’s data model prevents subtle bugs.

```
a = [1, 2, 3]
b = a
c = [1, 2, 3]

print(a == c)  # True (same values)
print(a is c)  # False (different objects)
print(a is b)  # True (same object)

my_set = {1, 2, 3}
# my_set.add([4, 5])  # TypeError: unhashable list
my_set.add((4, 5))    # OK: tuple is hashable
```

Output:

```
True
False
True
```

`==` compares values; `is` compares identity. Mutable types (list, dict, set) are unhashable by default; immutable types (str, int, tuple) are hashable.

Iterators and Generators — Lazy iteration

Iterators let you process items without storing everything in memory.

```
def countdown(n):
    while n > 0:
        yield n
        n -= 1

for value in countdown(3):
    print(value)
```

Output:

```
3
2
1
```

`yield` produces values one at a time. This is memory-efficient and ideal for large or infinite sequences.

Comprehensions — List, dict, set, generator

Comprehensions are compact ways to build collections.

```
squares = [n * n for n in range(6)]
lookup = {n: n * n for n in range(6)}
evens = {n for n in range(10) if n % 2 == 0}
gen = (n * n for n in range(6))
```

Resulting values:

```
squares = [0, 1, 4, 9, 16, 25]
lookup = {0: 0, 1: 1, 2: 4, 3: 9, 4: 16, 5: 25}
```

```
evens = {0, 2, 4, 6, 8}
gen = <generator object ...>
```

Use list/dict/set comprehensions when you want a concrete collection. Use a generator expression when you want lazy evaluation.

Functions Advanced — Default args, keyword-only, decorators

Default args are evaluated once; keyword-only args improve clarity.

```
def greet(name, *, punctuation="!"):
    return f"Hello, {name}{punctuation}"

def logger(fn):
    def wrapper(*args, **kwargs):
        print("calling:", fn.__name__)
        return fn(*args, **kwargs)
    return wrapper

@logger
def add(a, b):
    return a + b
```

Output:

```
(no output; functions are defined)
```

The `*` makes `punctuation` keyword-only. Decorators wrap functions for logging, timing, caching, and access control.

Context Managers — with, **enter**, **exit**

Context managers ensure cleanup even on errors.

```
class ManagedFile:
    def __init__(self, path):
        self.path = path
        self.file = None
    def __enter__(self):
        self.file = open(self.path, "w", encoding="utf-8")
        return self.file
    def __exit__(self, exc_type, exc, tb):
        if self.file:
            self.file.close()

with ManagedFile("notes.txt") as f:
    f.write("safe write")
```

Output:

```
(no output; writes to notes.txt)
```

`with` calls `__enter__` at the start and `__exit__` at the end, even if an exception occurs.

Exceptions — Custom types and best practice

Raise specific exceptions and keep error handling tight.

```
class ParseError(Exception):  
    pass  
  
def parse_int(text):  
    try:  
        return int(text)  
    except ValueError as exc:  
        raise ParseError("not an int") from exc
```

Output:

```
(no output; functions are defined)
```

Use custom exceptions to signal specific failures. `raise ... from ...` preserves the original traceback for debugging.

Dataclasses — Cleaner data containers

Dataclasses reduce boilerplate for simple data objects.

```
from dataclasses import dataclass  
  
@dataclass  
class Point:  
    x: int  
    y: int  
  
p = Point(2, 3)  
print(p)
```

Output:

```
Point(x=2, y=3)
```

Dataclasses auto-generate `__init__`, `__repr__`, and comparisons unless you override them.

Typing — Hints for clarity and tooling

Type hints improve readability and enable static analysis.

```
from typing import Iterable

def total(values: Iterable[int]) -> int:
    return sum(values)
```

Output:

```
(no output; function is defined)
```

Hints are not enforced at runtime, but tools like `mypy` can check them.